

Universidad Gerardo Barrios
Facultad de Ciencia y Tecnología



Modulo:

Desarrollo de Aplicaciones Inteligentes

Nombre del proyecto:

Best.IA

Docente:

Marco Antonio Arévalo Zambrano

Presentado por:

PLACIDO ISMAEL LUNA ARGUETA SMIS071121

NASER AUDELI CLAROS RIVERA SMIS099121

SOFIA GISSELL HERNADEZ ASCENCIO SMIS015421

San Miguel, 1 de Agosto de 2026

DESCRIPCION

BEST.IA es una plataforma de acompañamiento emocional para estudiantes universitarios. El sistema permite recibir mensajes del usuario, detectar la emoción predominante y generar una respuesta empática orientada al bienestar emocional. El proyecto usa un frontend en Next.js y un microservicio de análisis emocional con FastAPI y PySentimiento.

RUTA ELEGIDA

La ruta elegida para el despliegue fue Docker.

Se implementó una arquitectura con dos servicios principales:

web: aplicación principal en Next.js.

emotion-service: microservicio Python con FastAPI para análisis emocional.

Esta arquitectura permite levantar el sistema de forma reproducible en cualquier entorno con Docker.

DOCKERFILE Y CONFIGURACIÓN

Se incluyó un Dockerfile para la app web y un Dockerfile.emotion para el microservicio de emociones, también se configuró docker-compose.yml para levantar ambos servicios en red interna.

DOCKERFILE WEB

Instala dependencias de Node.js.

Copia el código fuente.

Ejecuta npm run build.

Expone el puerto 3000.

```

1 FROM node:20-alpine AS base
2
3 WORKDIR /app
4
5 COPY package.json package-lock.json ./
6 RUN npm install
7
8 COPY . .
9
10 ARG NEXT_PUBLIC_SUPABASE_URL
11 ARG NEXT_PUBLIC_SUPABASE_ANON_KEY
12 ARG PYSENTIMIENTO_API_URL
13
14 ENV NEXT_PUBLIC_SUPABASE_URL=$NEXT_PUBLIC_SUPABASE_URL
15 ENV NEXT_PUBLIC_SUPABASE_ANON_KEY=$NEXT_PUBLIC_SUPABASE_ANON_KEY
16 ENV PYSENTIMIENTO_API_URL=$PYSENTIMIENTO_API_URL
17
18 RUN npm run build
19
20 EXPOSE 3000
21
22 CMD ["npm", "start"]

```

DOCKERFILE EMOTION

Usa Python 3.11.

Instala dependencias de FastAPI, Uvicorn y PySentimiento.

Expone el puerto 8000.

Ejecuta el servicio con Uvicorn.

```

1 FROM python:3.11-slim
2
3 WORKDIR /app
4
5 COPY requirements.txt .
6 RUN pip install --no-cache-dir --default-timeout=120 --retries 5 \
7     --extra-index-url https://download.pytorch.org/whl/cpu \
8     -r requirements.txt
9
10 COPY emotion_service.py .
11
12 EXPOSE 8000
13
14 CMD ["uvicorn", "emotion_service:app", "--host", "0.0.0.0", "--port", "8000"]

```

DOCKER-COMPOSE.YML

Levanta web y emotion-service.

Define variables de entorno.

Configura el healthcheck del microservicio.

Permite comunicación interna entre contenedores.

```
3  services:
4    web:
5      build:
6        context: .
7        dockerfile: Dockerfile
8      args:
9        NEXT_PUBLIC_SUPABASE_URL: ${NEXT_PUBLIC_SUPABASE_URL}
10       NEXT_PUBLIC_SUPABASE_ANON_KEY: ${NEXT_PUBLIC_SUPABASE_ANON_KEY}
11      ports:
12        - "3000:3000"
13      env_file:
14        - .env.local
15      depends_on:
16        emotion-service:
17          condition: service_healthy
18
19  emotion-service:
20    build:
21      context: .
22      dockerfile: Dockerfile.emotion
23    ports:
24      - "8000:8000"
25    healthcheck:
26      test: ["CMD-SHELL", "python -c \"import urllib.request; urllib.request.urlopen('http://localhost:8000/health')\""]
27      interval: 10s
28      timeout: 5s
29      retries: 20
30      start_period: 90s
```

DOCKERIGNORE

Se excluyeron archivos innecesarios del contexto de build para reducir peso y evitar errores.

```
1  .dockerignore
2  node_modules
3  .next
4  .git
5  .env.local
6  .env
7  venv
8  __pycache__
9  *.pyc
10 tests
11 docs
12 .github
```

Archivos que se excluyen

VALIRABLES DE ENTORNO

VARIABLE	USO
NEXT_PUBLIC_SUPABASE_URL	URL del proyecto Supabase
NEXT_PUBLIC_SUPABASE_ANON_KEY	Clave pública de Supabase
PYSENTIMIENTO_API_URL	URL interna del microservicio de emociones
GOOGLE_API_KEY	Clave para Gemini

EVIDENCIA DE CONSTRUCCIÓN Y EJECUCIÓN

Durante la implementación se realizaron pruebas de construcción con Docker compose, se corrigieron errores de dependencias, variables de entorno y tiempo de descarga de librerías pesadas como torch.

Evidencias observadas:

Build exitoso del contenedor emotion-service.

Arranque correcto del servicio FastAPI.

Arranque correcto del frontend Next.js.

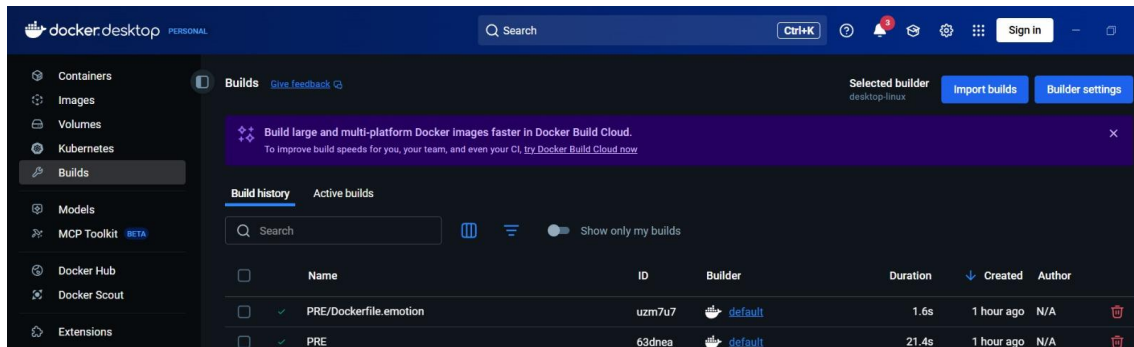
Comunicación entre contenedores mediante red de Docker.

```
PS C:\Users\placi\Desktop\PRE> docker compose up --build
=> => unpacking to docker.io/library/pre-web:latest
=> [web] resolving provenance for metadata file
[+] Running 5/5
✓ pre-emotion-service      Built
✓ pre-web                  Built
✓ Network pre_default      Created
✓ Container pre-emotion-service-1 Created
✓ Container pre-web-1      Created
Attaching to emotion-service-1, web-1
emotion-service-1 | Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable
.
Loading weights: 100%|██████████| 201/201 [00:00<00:00, 1182.08it/s]
emotion-service-1 | INFO:      Started server process [1]
```

PRUEBA DEL ENDPOINT /HEALTH

El servicio de encuentra saludable

```
emotion-service-1 | INFO:      127.0.0.1:38146 - "GET /health HTTP/1.1" 200 OK
emotion-service-1 | INFO:      127.0.0.1:60256 - "GET /health HTTP/1.1" 200 OK
emotion-service-1 | INFO:      127.0.0.1:57304 - "GET /health HTTP/1.1" 200 OK
emotion-service-1 | INFO:      127.0.0.1:55258 - "GET /health HTTP/1.1" 200 OK
emotion-service-1 | INFO:      127.0.0.1:42756 - "GET /health HTTP/1.1" 200 OK
emotion-service-1 | INFO:      127.0.0.1:51292 - "GET /health HTTP/1.1" 200 OK
emotion-service-1 | INFO:      127.0.0.1:49080 - "GET /health HTTP/1.1" 200 OK
emotion-service-1 | INFO:      127.0.0.1:42776 - "GET /health HTTP/1.1" 200 OK
emotion-service-1 | INFO:      127.0.0.1:41746 - "GET /health HTTP/1.1" 200 OK
emotion-service-1 | INFO:      127.0.0.1:53266 - "GET /health HTTP/1.1" 200 OK
```

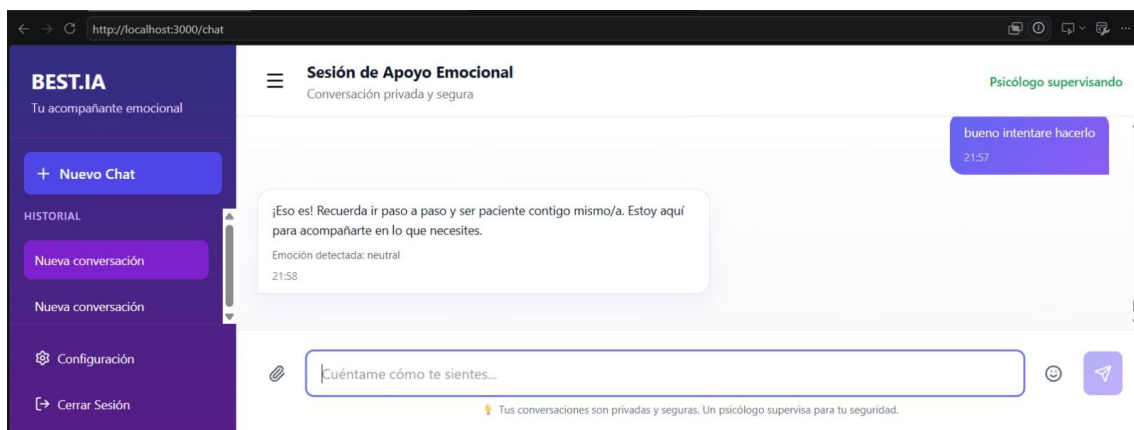


Los contenedores están arriba sin ningún error.

PRUEBA DEL ENDPOINT PRINCIPAL

El endpoint principal del proyecto es la interfaz del chat:

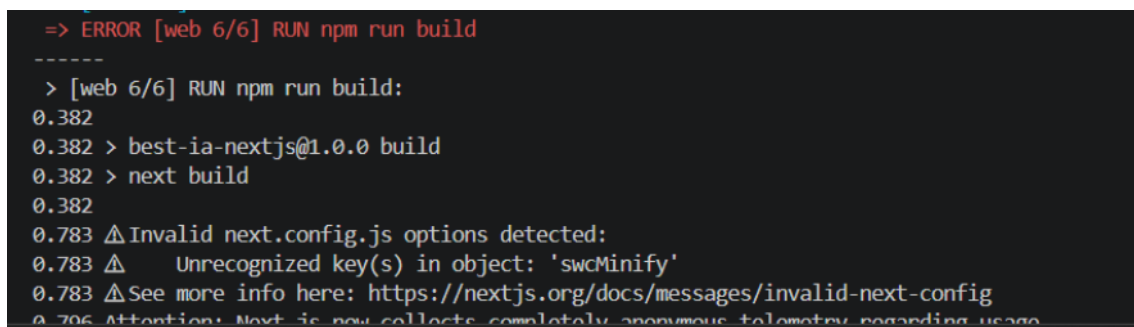
La interfaz carga correctamente y permite enviar mensajes al sistema, también se verificó que el frontend muestra la conversación y que el servicio responde con una salida emocional.



ERRORES Y CAUSAS

Error durante docker build (next.js)

Causa: en tsconfig.json, y next build ejecuta un chequeo estricto de TypeScript que next dev no aplica.



Error: variables de entorno no disponibles en build de Docker que corresponden a supabase

Causa: La APIKEY de entorno no se encontraban en el archivo .env local

```
7.510 Type error: 'data' is declared but its value is never read.
7.510
7.510 46 |
7.510 47 |     try {
7.510 > 48 |         const { data, error: signUpError } = await supabase.auth.signUp({
7.510    |         ^
7.510 49 |         email: formData.email,
7.510 50 |         password: formData.password,
7.510 51 |         options: {
7.585 Next.js build worker exited with code: 1 and signal: null
```

Error: supabaseUrl is required al prerenderizar /auth/login dentro del contenedor.

Causa: Next.js necesita las variables NEXT_PUBLIC_ en tiempo de build para generar páginas estáticas, pero .env.local está excluido del contexto de Docker por seguridad (.dockerignore).

```
7.510 Type error: 'data' is declared but its value is never read.
7.510
7.510 46 |
7.510 47 |     try {
7.510 > 48 |         const { data, error: signUpError } = await supabase.auth.signUp({
7.510    |         ^
7.510 49 |         email: formData.email,
7.510 50 |         password: formData.password,
7.510 51 |         options: {
7.585 Next.js build worker exited with code: 1 and signal: null
```

PLAN DE INFRAESTRUCTURA MÍNIMA

La infraestructura mínima propuesta es:

- 1 contenedor para la app web.
- 1 contenedor para el servicio de emociones.
- Red interna de Docker para comunicación entre servicios.
- Variables de entorno cargadas desde .env.local.
- Healthcheck para validar disponibilidad del microservicio.

ESTIMACIÓN DE COSTOS INICIALES

Para una versión mínima local o académica, el costo inicial puede ser bajo o nulo si se ejecuta en equipo personal.

Supuestos:

- Uso local con Docker: sin costo adicional.
- Hosting futuro en PaaS de bajo costo: entre 5 y 20 USD mensuales.

- Base de datos o servicios externos: dependerá del uso de Supabase y Gemini.

Estimación inicial:

- Desarrollo local: 0 USD.
- Despliegue básico en nube: 5–20 USD/mes.
- Crecimiento con más usuarios: costo variable según tráfico y uso de APIs.

RIESGOS TÉCNICOS PENDIENTES

1. El microservicio de emociones depende de modelos pesados que tardan en cargar.
2. El uso de APIs externas puede introducir latencia.
3. Si cambia la configuración de variables de entorno, el sistema puede fallar al arrancar.
4. La estabilidad del despliegue depende de que los contenedores se sincronicen correctamente.

Enlace a Repositorio:

<https://github.com/ismael-15/BEST.IA.git>